

CHATGATE MIDDLEWARE DESIGN BRIEF

Create an Express application built as part of the Chatgate framework using the Botpress SDK, Nodejs, Botpress Cloud, Facebook Pages API, X API and Instagram.

Define a set of RESTful API routes for managing agents, handoffs, comments, and conversations within a bot context.

Prepared by: Michael Mukwekezeke

1. Imports

lodash: A utility library for JavaScript which offers functions for working with arrays, numbers, objects, etc.

LRU: A cache implementation (Least Recently Used) to temporarily store handoff data for quick retrieval.

ms: A library used to convert time strings into milliseconds.

Config, MODULE_NAME, StateType, IAgent, IHandoff: These are custom modules or types defined elsewhere, which likely represent configuration settings, constants, and TypeScript interfaces that help maintain type safety.

`extendAgentSession` and `measure`: Utility functions for agent session management and performance measuring.

Repository: A class that abstracts the data layer to interact with the database or data store.

Socket: A module to manage real-time socket connections for communicating events.

2. Helper Functions

2.1. HITL Status Updates

updateHitlStatus: This function updates the event object with details about the agent handling a handoff. If the event is of type 'hitlnext', it extracts the exitType and agentName, storing them in the state.temp object of the event.

2.2. Middleware Registration

registerMiddleware: This is the main function responsible for registering middleware in the event system. Here's what it does in detail:

Initialize Caches and Repositories:

Creates an LRU cache to store handoff IDs temporarily with a maximum size of 1000 entries, expiring after 1 day.

Initializes a Repository instance for data retrieval and manipulation.

Creates a Socket instance for real-time communication.

2.3 Event Handling Functions

Defines several functions to handle various incoming events.

2.3.1. Event Allocation

pipeEvent

This function sends an incoming event to the specified destination (either user or agent). The debug logger records the event piping action.

2.3.2. Cache Functions

handoffCacheKey: Generates a unique key for caching based on the bot ID and thread ID.

getCachedHandoff: Retrieves a handoff ID from the cache using the generated key.

cacheHandoff: Stores a handoff ID in the LRU cache and logs the action.

2.3.3. Expired Handoff

expireHandoff: Removes a handoff entry from the cache, logging the action.

2.4 Handling Incoming Events

handleIncomingFromUser: This function handles events when they originate from a user. If the handoff status is 'assigned', it maps the agent ID back to a user ID and pipes the event to the appropriate destination.

If the handoff is not assigned, it constructs a partial event with relevant information and sends updates about the handoff to the frontend in real-time.

handleIncomingFromAgent: This function processes events from agents. It retrieves agent details (e.g., name) and formats the event payload accordingly before sending it to the user. It also extends the agent's session to keep them connected with the system.

incomingHandler: This middleware function processes incoming events.

- **Event Type Verification:** The handler checks if the event is of type 'text'. If not, it calls the next function to skip further processing.
- **Handoff ID Lookup:** The function checks for an existing handoff ID associated with the bot and thread ID. If none exists, it calls next to terminate the process for this event.
- **Retrieving Handoff Details:** When a handoff ID is available, the details of the handoff are retrieved from the repository.
- **Source Event Check:** It determines the origin of the incoming event (either from the user or the agent).
- **Processing Condition:** Depending on the origin, it processes the event using the appropriate handler (`handleIncomingFromUser` or `handleIncomingFromAgent`).
- **Session Management:** The event is flagged as paused, and the handoff ID is attached to the event for reference. Regardless of the message source, the event is marked with `isPause: true` to indicate that the session is currently paused, and assigns the current handoff ID for further reference.
- **Control Flow:** It finally invokes `next()` to continue to the next middleware, if applicable.

2.5. Caching Setup

This function:

- Fetches all active handoffs and caches them, allowing for quick access during incoming message handling.
- It ensures that both the agent and user thread ID references for each handoff are cached, facilitating smooth message routing.

3. Performance Measurement and Middleware Registration

3.1 Performance

Debugging Performance: If debugging is enabled, the warmup process is measured for performance.

Function Broadcasting: Functions to cache and expire handoffs are made available across distributed instances of Chatgate for consistency and scalability.

Middleware Registration: The incoming event handler is registered with Chatgate, specifying its execution order and purpose.

3.2 Unregistering Middleware

This function is responsible for cleaning up by unregistering the incoming middleware, ensuring that no references to it remain when the module is deactivated.

3.3 Exporting Functions

The module exports functions for registering and unregistering the middleware. This allows integration into the wider Chatgate framework, enabling or disabling the handler as required.

4. Table Schema

The provided code snippet defines the schemas for two database tables, Handoff and Comment, in a Botpress application. Below is a thorough description of each table, including the purpose of each column and the relationships between them.

4.1. Handoff Table

Table Name: Handoff

The Handoff table is designed to manage the handoff process between users and agents within a conversational interface. This table keeps track of the interactions, the status of those interactions, and relevant metadata.

Columns:

id (increments):

Type: Integer (Auto-incrementing)

Description: A unique identifier for each handoff record. It serves as the primary key for the table.

botId (string):

Type: String

Description: The identifier of the bot that is handling the conversation. This field is mandatory and helps associate the handoff with a specific bot instance.

userId (string):

Type: String

Description: The unique identifier of the user involved in the handoff. This field is mandatory, ensuring that each handoff can be traced back to a specific user.

agentId (string):

Type: String (optional)

Description: The identifier of the agent (human or bot) that the handoff is directed to. This field can be null if the handoff has not yet been assigned to an agent.

userThreadId (string):

Type: String

Description: The identifier for the user's thread of conversation. This is mandatory and allows the system to track the context of the conversation for that specific user.

userChannel (string):

Type: String

Description: The communication channel used by the user (e.g., web, mobile, etc.). This field is mandatory and helps identify the platform through which the user is interacting.

agentThreadId (string):

Type: String (optional)

Description: The identifier for the agent's thread of conversation. This can be null if the handoff has not yet been assigned to an agent.

status (string):

Type: String

Description: The current status of the handoff (e.g., 'pending', 'assigned', 'resolved'). This field is mandatory and is crucial for managing the handoff workflow.

tags (json):

Type: JSON

Description: A field for storing additional metadata in a flexible format. This could include custom tags or labels relevant to the handoff.

assignedAt (dateTime):

Type: DateTime (optional)

Description: The timestamp indicating when the handoff was assigned to an agent. This field can be null if the handoff has not yet been assigned.

resolvedAt (dateTime):

Type: DateTime (optional)

Description: The timestamp indicating when the handoff was resolved. This field can be null if the handoff is still open.

createdAt (dateTime):

Type: DateTime

Description: The timestamp for when the handoff record was created. This is mandatory for auditing purposes.

updatedAt (dateTime):

Type: DateTime

Description: The timestamp for when the handoff record was last updated. This is mandatory and is useful for tracking changes to the record.

Constraints:

Unique Constraint: The id column is unique, ensuring no two records can have the same identifier.

Indexing: The botId column is indexed to improve query performance when filtering handoffs by bot.

4.2. Comment Table

Table Name: Comment

The Comment table is designed to store comments or messages associated with a specific handoff. This allows for tracking additional context or notes related to the handoff process.

Columns:

id (increments):

Type: Integer (Auto-incrementing)

Description: A unique identifier for each comment record. It serves as the primary key for the table.

handoffId (integer):

Type: Integer

Description: A foreign key referencing the id of the Handoff table. This field is mandatory and establishes a relationship between comments and the handoff they are associated with. If the handoff is deleted, the comments associated with it will also be deleted due to the onDelete('CASCADE') constraint.

threadId (string):

Type: String

Description: The identifier for the conversation thread in which the comment was made. This field is mandatory and helps maintain context for the comment.

agentId (string):

Type: String

Description: The identifier of the agent who made the comment. This field is mandatory, ensuring that each comment can be attributed to a specific agent.

content (text):

Type: Text (optional)

Description: The actual content of the comment. This field can be null if no content is provided.

createdAt (dateTime):

Type: DateTime

Description: The timestamp for when the comment record was created. This is mandatory for auditing purposes.

updatedAt (dateTime):

Type: DateTime

Description: The timestamp for when the comment record was last updated. This is mandatory and useful for tracking changes to the comment.

Constraints:

Unique Constraint: The id column is unique, ensuring no two records can have the same identifier.

Indexing: The handoffId and agentId columns are indexed to improve query performance when filtering comments by handoff or agent.

4.3 Summary of Relationships

The Handoff table serves as a parent table, while the Comment table is a child table that references the Handoff table through the handoffId foreign key. This relationship allows for the organization of comments related to specific handoffs, facilitating better tracking and management of interactions within the Botpress system.

This schema design supports efficient data retrieval and management, ensuring that both handoff processes and associated comments are well-structured and easily accessible. If you have any further questions or need additional details, feel free to ask!